DevOps 2TIN

Chapter 5b

CI in Github Actions

DE HOGESCHOOL MET HET NETWERK

Elfde-Liniestraat 24, 3500 Hasselt, www.pxl.be

CI in Github Actions

Secrets
unit testing
artifacts
Runners

HOGESCHOOL PXL

Wat is een Pipeline?

Definitie “deployment pipeline”:

(first defined by Jez Humble and David Farley in their book *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*)

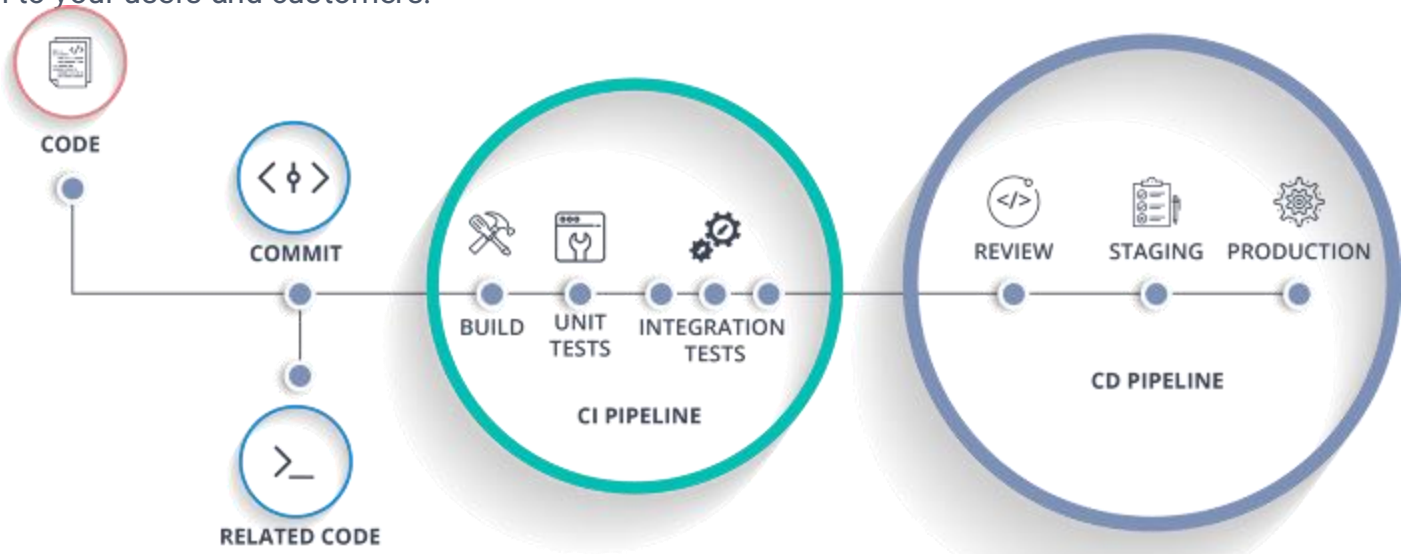
It ensures that all code checked in to version control is automatically built and tested in a production-like environment.

Sleutelwoorden:

- Alle Code
- Versiebeheer
- **Automatisch gebouwd**
- Automatisch getest
- Productie-waardige omgeving

Recap: CI/CD pipeline

A ***continuous delivery (CD) pipeline*** is an automated expression of your process for getting software from version control right through to your users and customers.

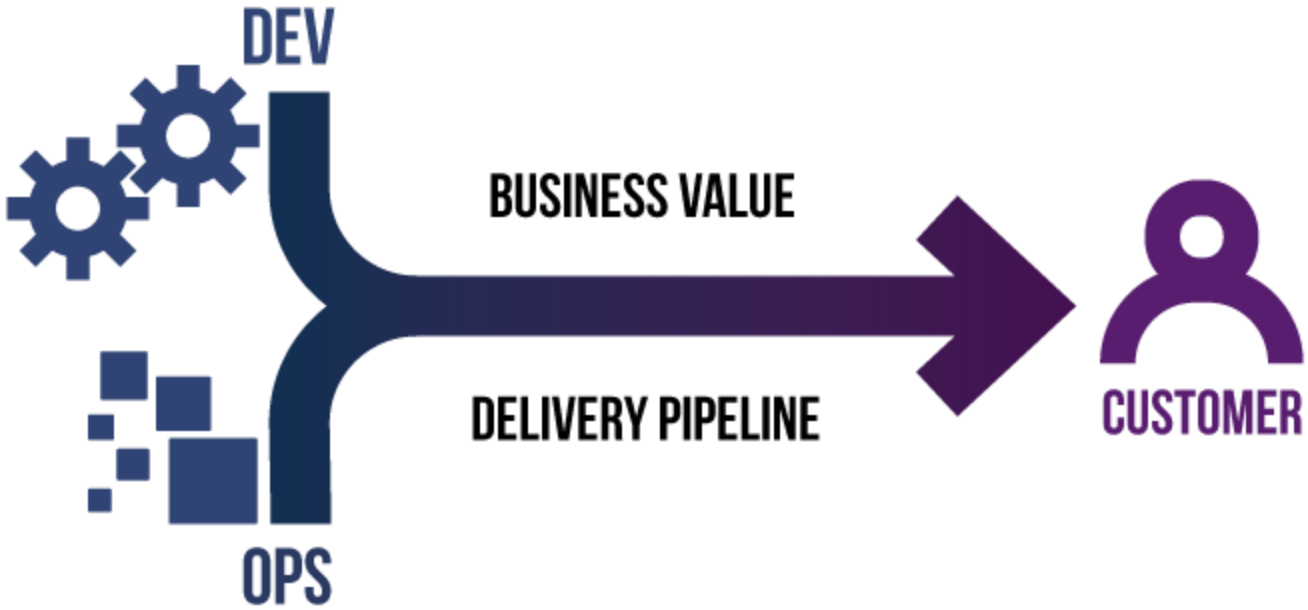


Every change to your software (committed in source control) goes through a complex process on its way to being released. This process involves building the software in a reliable and repeatable manner, as well as progressing the built software (called a "build") through multiple stages of testing and deployment.

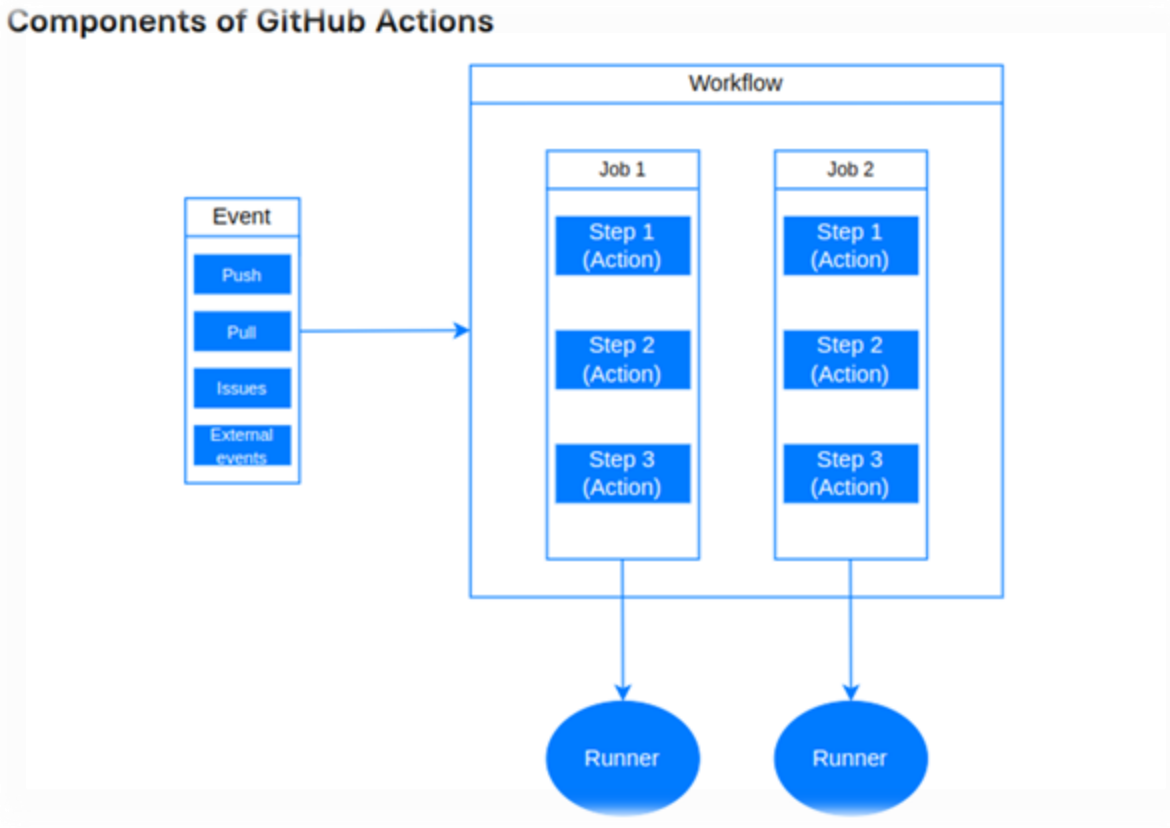
THE FIRST WAY

CREATE A DELIVERY SYSTEM

PXL DIGITAL



Recap



PXL DIGITAL

Recap

Event

```
on: push
```

Runner: `ubuntu-latest`

Job: `check-bats-version`

Step 1: Check out repository

```
- uses: actions/checkout@v3
```

Step 2: Install Node.js

```
- uses: actions/setup-node@v3
```

Step 3: Install bats

```
- run: npm install -g bats
```

Step 4: Run bats

```
- run: bats -v
```

Github Actions: Environment variables

- Github voorziet een hele hoop [systeemvariabelen]() over de context van je workflow run
- Alle info over :
  - workflow run
  - Repository
  - Job
  - Environment & secrets
  - ...

```yaml
steps:
  - name: Who triggered the workflow
    run: echo "Pipeline started by ${{ github.actor }}"

  - name: Repository info
    run: |
      echo "Repository: ${{ github.repository }}"
      echo "Default branch: ${{ github.event.repository.default_branch }}"
      echo "Repo URL: ${{ github.event.repository.html_url }}"

  - name: Runner info
    run: |
      echo "Runner OS: ${{ runner.os }}"
      echo "Runner architecture: ${{ runner.arch }}"

  - name: Job info
    run: |
      echo "Job name: ${{ github.job }}"
      echo "Workflow: ${{ github.workflow }}"
      echo "Run number: ${{ github.run_number }}"
```

Github Actions: Environment variables

- Daarnaast is het mogelijk om zelf (environment) variabelen aan te maken
- Om variabelen aan te maken maken we gebruik van het `env` keyword
  - Dit kan je op verschillende plaatsen gebruiken

```yaml
on:
  workflow_dispatch

env:
  DAY_OF_WEEK: Monday

jobs:
  greeting_job:
    runs-on: ubuntu-latest
    env:
      Greeting: Hello
    steps:
      - name: "Say Hello Mona it's Monday"
        run: echo "$Greeting $First_Name. Today is $DAY_OF_WEEK!"
        env:
          First_Name: Mona
```

Github Actions: Secret management

- Wat als we gevoelig informatie in onze workflow willen gebruiken?
  - Username, passwords, tokens, private keys, …
- **In plaintext in onze YML files = security risk!**

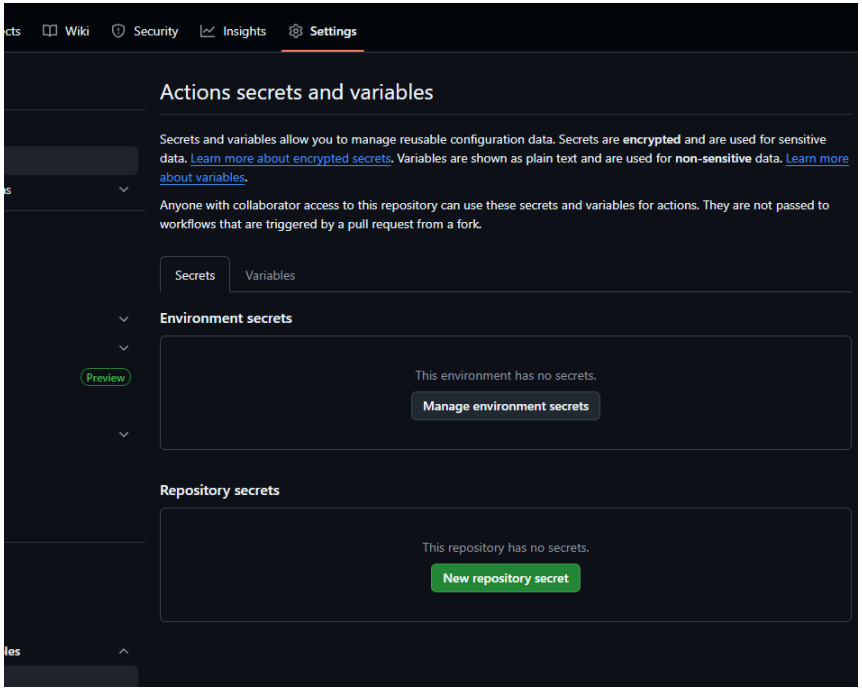
Github Actions: secrets

voorbeeld: verbinden met een VM in de cloud om onze code te deployen:

```
jobs:
deploy:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v2
- name: Deploy to EC2
  env:
    PRIVATE_KEY: "----BEGIN OPENSSH PRIVATE KEY----B3BlbnNzaC...AI4fVFeKj9AliW2Jgaxeg==----END OPENSSH
PRIVATE KEY----"

  HOST: 52.48.75.998
  USER: ec2-user
  run: |
  echo "$PRIVATE_KEY" > github-ec2.pem && chmod 600 github-ec2.pem
  ssh -o StrictHostKeyChecking=no -i github-ec2.pem ${USER}@${HOST} '
```

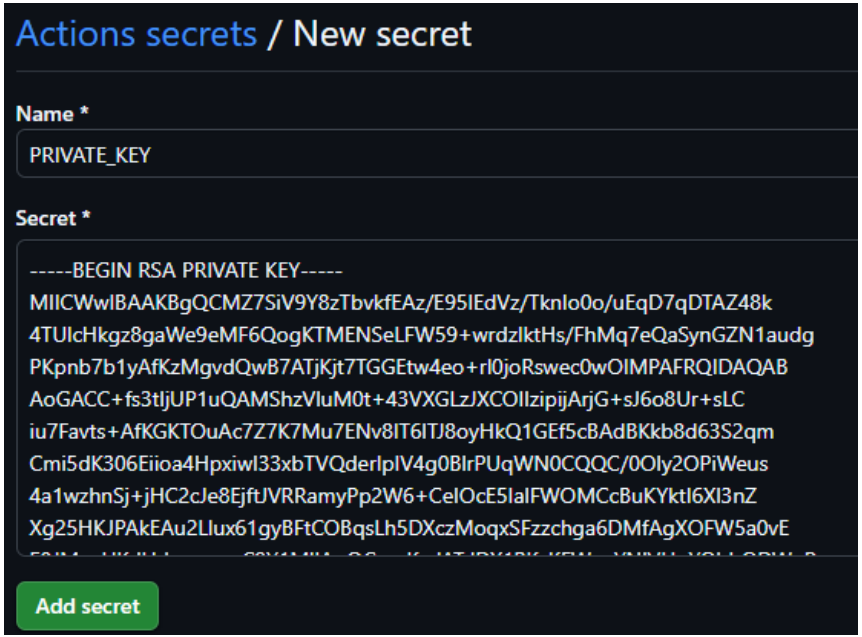
Github Actions: Secret management

- Gebruik maken van ingebouwde secret store van Github
- Kluis die onze gevoelige data beschermt en injecteert waar nodig
- Admin rechten op repository nodig



Github Actions: Secret management

- Gebruik maken van ingebouwde secret store van Github
- Kluis die onze gevoelige data beschermt en injecteert waar nodig
- Admin rechten op repository nodig



Github Actions: Secret management

- In een workflow file zijn secrets aanspreekbaar via ${{ secrets.NAAMSECRET }}
- Integratie via environment variables

```
jobs:
 deploy:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v2
    - name: Deploy to EC2
  env:
    PRIVATE_KEY: ${{ secrets.EC2_PRIVATE_KEY }}
    HOST: ${{ secrets.EC2_HOST }}
    USER: ${{ secrets.EC2_USER }}
  run: |
    echo "$PRIVATE_KEY" > github-ec2.pem && chmod 600 github-ec2.pem
    ssh -o StrictHostKeyChecking=no -i github-ec2.pem ${USER}@${HOST} '
```

Github Actions: Continious integration

- CI is niet zwart wit:
  - Elke technologiestack heeft zijn eigen stappen en tooling
  - Elk bedrijf hecht meer of minder waarde aan bepaalde stappen
  - Doel blijft hetzelfde: hoge velocity & snelle/eenvoudige deployments
- Niet nodig om het wiel opnieuw uit te vinden
  - Maak gebruik van de Github actions marketplace
  - Online veel voorbeelden & praktische cases te vinden voor populaire technologiestacks zoals Java, .NET, NodeJS, Python, React, Angular, Vue, ...

Code	Dependencies	(unit)test	build	archive artifact
CI				

Github Actions: pipeline – Installing dev dependencies

- Vaak gebruiken we tools om 3rd party packages & modules te installeren:
  - Npm voor NodeJS
  - Pip voor Python
  - Composer voor PHP
  - …
- Gebruik Marketplace plugins voor integratie van deze tools waar mogelijk!

Github Actions: pipeline – Installing dev dependencies

- NodeJS voorbeeld

```yaml
10        steps:
11          - name: Checkout code
12            uses: actions/checkout@v4
13
14          - name: Setup Node.js
15            uses: actions/setup-node@v4
16            with:
17              node-version: '20'
18
19          - name: Install dependencies
20            run: npm install
21
22          - name: List files
23            run: ls -alh
```

Install dependencies

```
1   ▶ Run npm install
4
5   added 23 packages, and audited 24 packages in 4s
6
7   6 packages are looking for funding
8     run `npm fund` for details
9
10  found 0 vulnerabilities
```

Github Actions: pipeline - Unit testing

- Hangt af van de testrunner van de applicatie
  - Java => JUnit
  - NodeJS => Jest
- Vaak integratie vanuit andere talen naar JUnit
- Testrunners hebben verschillende export mogelijkheden voor rapportering

Github Actions: pipeline - Unit testing

- Testen runnen a.d.h.v. testunner
- Default output in console
  - Minder overzichtelijk

```
unittests maven
1647  1
1648  2
1649  6
1650  [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0 s - in FatorialTest
1651  [INFO] Running AbsolutoTest
1652  2
1653  2
1654  [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0 s - in AbsolutoTest
1655  [INFO]
1656  [INFO] Results:
1657  [INFO]
1658  [INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0
1659  [INFO]
1660  [INFO] ------------------------------------------------------------------------
1661  [INFO] BUILD SUCCESS
1662  [INFO] ------------------------------------------------------------------------
1663  [INFO] Total time:  8.490 s
1664  [INFO] Finished at: 2025-09-26T09:25:51Z
```

```yaml
steps:
- uses: actions/checkout@v4
- name: Set up JDK 17
  uses: actions/setup-java@v4
  with:
    java-version: '17'
    distribution: 'temurin'
    cache: maven
- name: unittests maven
  run: mvn test
```

Github Actions: pipeline - Unit testing

- Rapportering uit de console trekken a.d.h.v. [test-reporter]
  - Integratie met Github UI
  - Link aan build poging

Triggered via push 2 minutes ago
d-ries pushed e921004 master

Status: Success
Total duration: 20s
Artifacts: –

maven.yml
on: push

build 16s

build summary

20 passed, 0 failed and 0 skipped tests 20 passed

▼ Expand for details

Report	Passed	Failed	Skipped	Time
target/surefire-reports/TEST-AbsolutoTest.xml	2			1ms
target/surefire-reports/TEST-BigBrotherTest.xml	8			5ms
target/surefire-reports/TEST-BookTest.xml	1			0ms
target/surefire-reports/TEST-ContaCorrenteTest.xml	3			1ms
target/surefire-reports/TEST-ContaMockTest.xml	4			1ms
target/surefire-reports/TEST-FatorialTest.xml	2			0ms

target/surefire-reports/TEST-AbsolutoTest.xml

2 tests were completed in 1ms with 2 passed, 0 failed and 0 skipped.

Github Actions: pipeline - Unit testing

- Rapportering uit de console trekken a.d.h.v. [test-reporter]
  - Integratie met Github UI
  - Link aan build poging

```yaml
    steps:
    - uses: actions/checkout@v4
    - name: Set up JDK 17
      uses: actions/setup-java@v4
      with:
        java-version: '17'
        distribution: 'temurin'
        cache: maven
    - name: unittests maven
      run: mvn test
    - name: check files
      run: ls -alh; ls -alh ./target/surefire-reports
    - name: Test Report
      uses: dorny/test-reporter@v2
      with:
          name: JUnit Tests
          path: target/surefire-reports/TEST-*.xml
          reporter: java-junit
```

Github Actions: pipeline - Unit testing

- Rapportering uit de console trekken a.d.h.v. [test-reporter]
  - Integratie met Github UI
  - Link aan build poging

Triggered via push 2 minutes ago
d-ries pushed e921004 master

Status: Success
Total duration: 20s
Artifacts: –

maven.yml
on: push

build 16s

build summary

20 passed, 0 failed and 0 skipped tests 20 passed

▼ Expand for details

Report	Passed	Failed	Skipped	Time
target/surefire-reports/TEST-AbsolutoTest.xml	2			1ms
target/surefire-reports/TEST-BigBrotherTest.xml	8			5ms
target/surefire-reports/TEST-BookTest.xml	1			0ms
target/surefire-reports/TEST-ContaCorrenteTest.xml	3			1ms
target/surefire-reports/TEST-ContaMockTest.xml	4			1ms
target/surefire-reports/TEST-FatorialTest.xml	2			0ms

target/surefire-reports/TEST-AbsolutoTest.xml

2 tests were completed in 1ms with 2 passed, 0 failed and 0 skipped.

Github actions - pipeline – Build / Bundle

- Builden van de applicatie hangt af van de technologiestack
  - Voor onze NodeJS applicatie vaak een zip van de nodige bestanden
    - Kritisch nadenken over wat nodig is voor production
- Docker!

Github actions - pipeline - Artifacts

- Na het uitvoeren van je workflow run wordt alle data verwijderd
- Artifacts zijn bestanden die je wil bewaren na het uitvoeren van je workflow run
    - Uitvoerbare file(s) van je applicatie
    - Logs
    - Test resultaten
    - Rapporten

Jenkins - pipeline - Artifacts

- Na het uitvoeren van je workflow run wordt alle data verwijderd
- Artifacts zijn bestanden die je wil bewaren na het uitvoeren van je workflow run

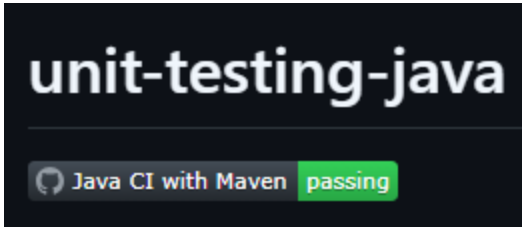
```yaml
- name: Archive production artifacts
  uses: actions/upload-artifact@v4
  with:
    name: app-package
    path: |
      target/*.jar
```

Artifacts
Produced during runtime

Name	Size	Digest
app-package	5.42 KB	sha256:889b6c77ec54e2e0bc3f9bacf04c78512780d4595f42c8e35a4887c8976ae70a

Extra integraties: Readme CI information

- Feedback over CI workflow kan rechtstreeks geïntegreerd worden in markdown files van de repository zoals README.md



- Voorzien van volgende URL in markdown file:

![CI](https://github.com/<OWNER>/<REPO>/actions/workflows/<WORKFLOW_FILE>/badge.svg)

Extra integraties: Publishing artifacts

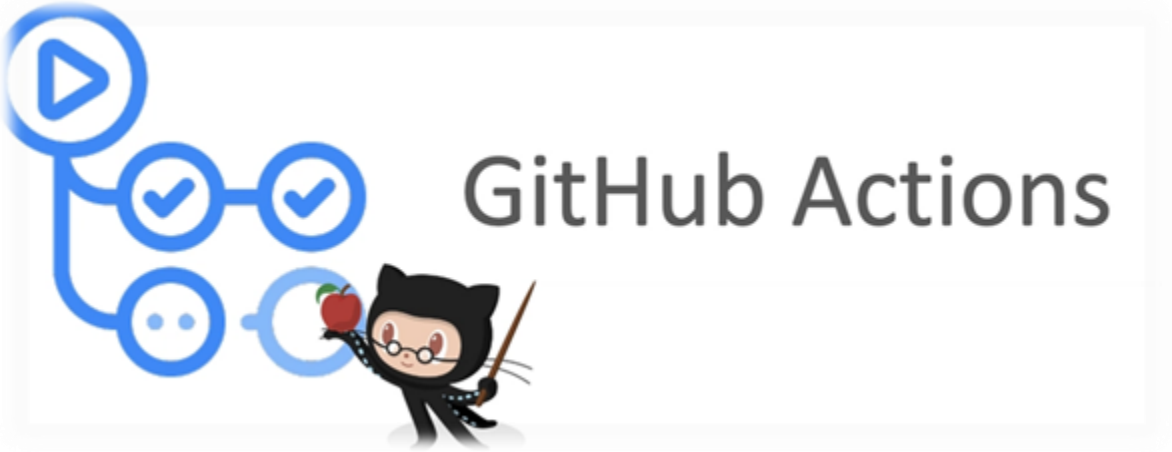
- Vaak worden artifacts uit de GH Actions flow gehaald en in een extern systeem gestoken
- Vaak Docker containers met tags die teruglinken aan workflow runs / commits
- Integratie met Dockerhub of Github container registry
    - Authenticatie a.d.h.v. Github secrets!
- Dockerfile moet aanwezig zijn in repository van de applicatie

Extra integraties: Publishing artifacts

```yaml
steps:
  - name: Check out the repo
    uses: actions/checkout@v5

  - name: Log in to Docker Hub
    uses: docker/login-action@f4ef78c080cd8ba55a85445d5b36e214a81df20a
    with:
      username: ${{ secrets.DOCKER_USERNAME }}
      password: ${{ secrets.DOCKER_PASSWORD }}

  - name: Extract metadata (tags, labels) for Docker
    id: meta
    uses: docker/metadata-action@9ec57ed1fcdbf14dcef7dfbe97b2010124a938b7
    with:
      images: my-docker-hub-namespace/my-docker-hub-repository

  - name: Build and push Docker image
    id: push
    uses: docker/build-push-action@3b5e8027fcad23fda98b2e3ac259d8d67585f671
    with:
      context: .
      file: ./Dockerfile
      push: true
      tags: ${{ steps.meta.outputs.tags }}
      labels: ${{ steps.meta.outputs.labels }}
```

Recap

PXL DIGITAL



Assignments

Lab 3 – CI in Github Actions

HOGESCHOOL PXL